

Infor Support Assistant

A retrieval-augmented support assistant for Infor WMS that runs entirely on one machine. Operators ask questions in plain language; the assistant answers from uploaded manuals, incident and defect exports, and recorded walkthroughs — grounded, source-cited, and confidence-gated, with no data leaving the host.

PROJECT Infor Support Assistant	CLASSIFICATION Proof of Concept	REVISION 0.1 – draft	DATE 2026-09-04
RUNTIME Single host / offline / CPU			

§1 What this is

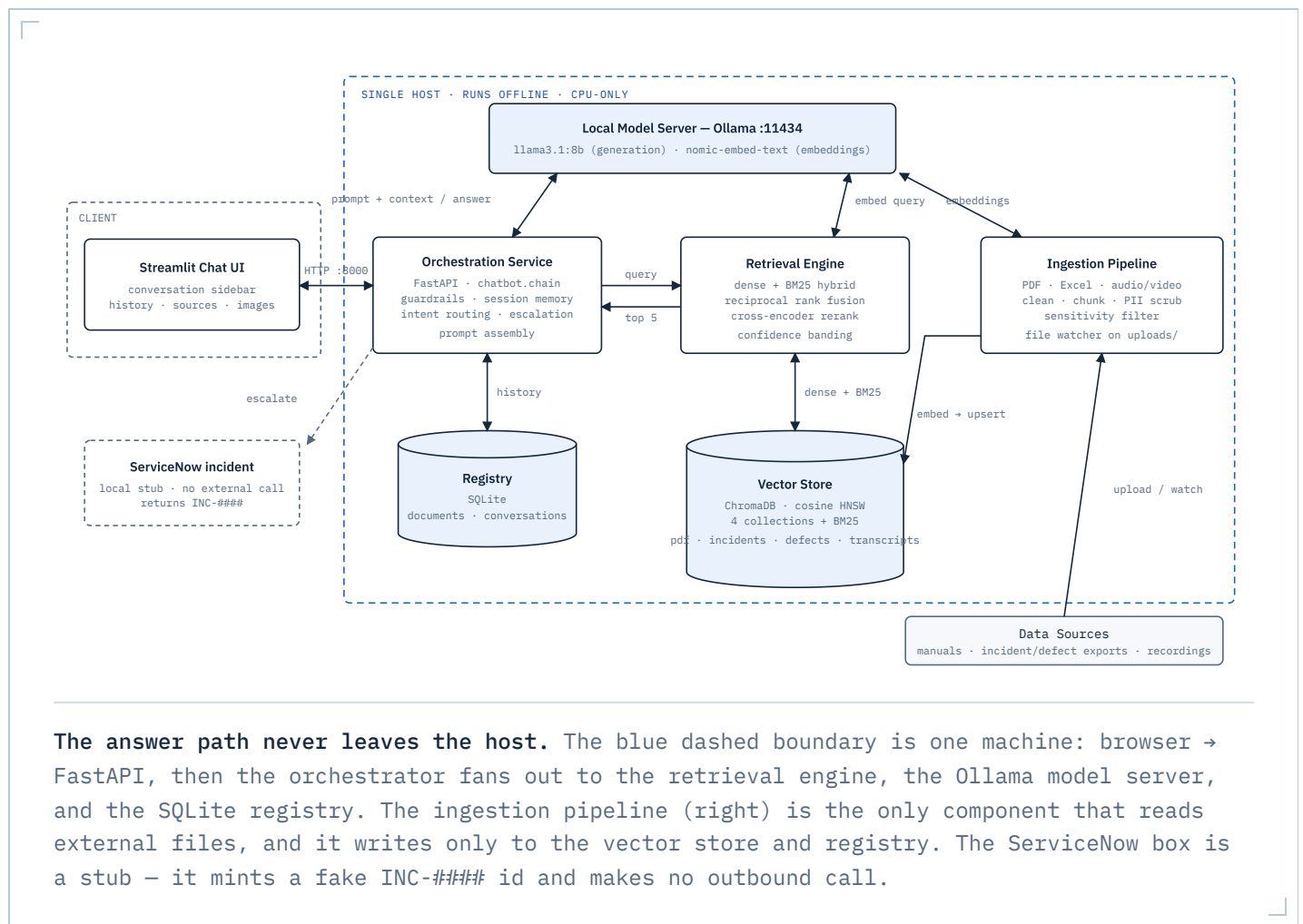
The assistant is a local **RAG** (retrieval-augmented generation) pipeline. Documents are ingested once into a vector store; at question time the system retrieves the passages most likely to answer the query, hands them to a local language model as context, and returns the model's answer alongside the exact sources it drew from.

Every moving part — the language model, the embedding model, the vector store, the reranker, the PII scrubber — runs on the same host as the API. There is no external API key, no cloud inference, and no outbound request in the answer path. That constraint shapes every decision below.

Design principles

- ❑ **Offline by construction.** Ollama serves generation and embeddings from `localhost:11434`; ChromaDB, BM25, the cross-encoder and spaCy all run in-process. The only network dependency is a one-time model download.
- ❑ **Answer only from what was uploaded.** The model is instructed to use retrieved context exclusively and to name sources by their real file names. A low-confidence retrieval routes to escalation rather than a guess.
- ❑ **Guarded on both sides.** Input is checked for injection and abuse; output is scanned for leaked PII, internal identifiers, and fabricated error numbers before it reaches the user.
- ❑ **Confidence is explicit.** Each answer carries a band — high, medium, or low — derived from the reranker score of the top passage.
- ❑ **Ingestion is hands-off.** A file dropped into `data/uploads/` is picked up, routed by type, cleaned, scrubbed, embedded and indexed without operator action.

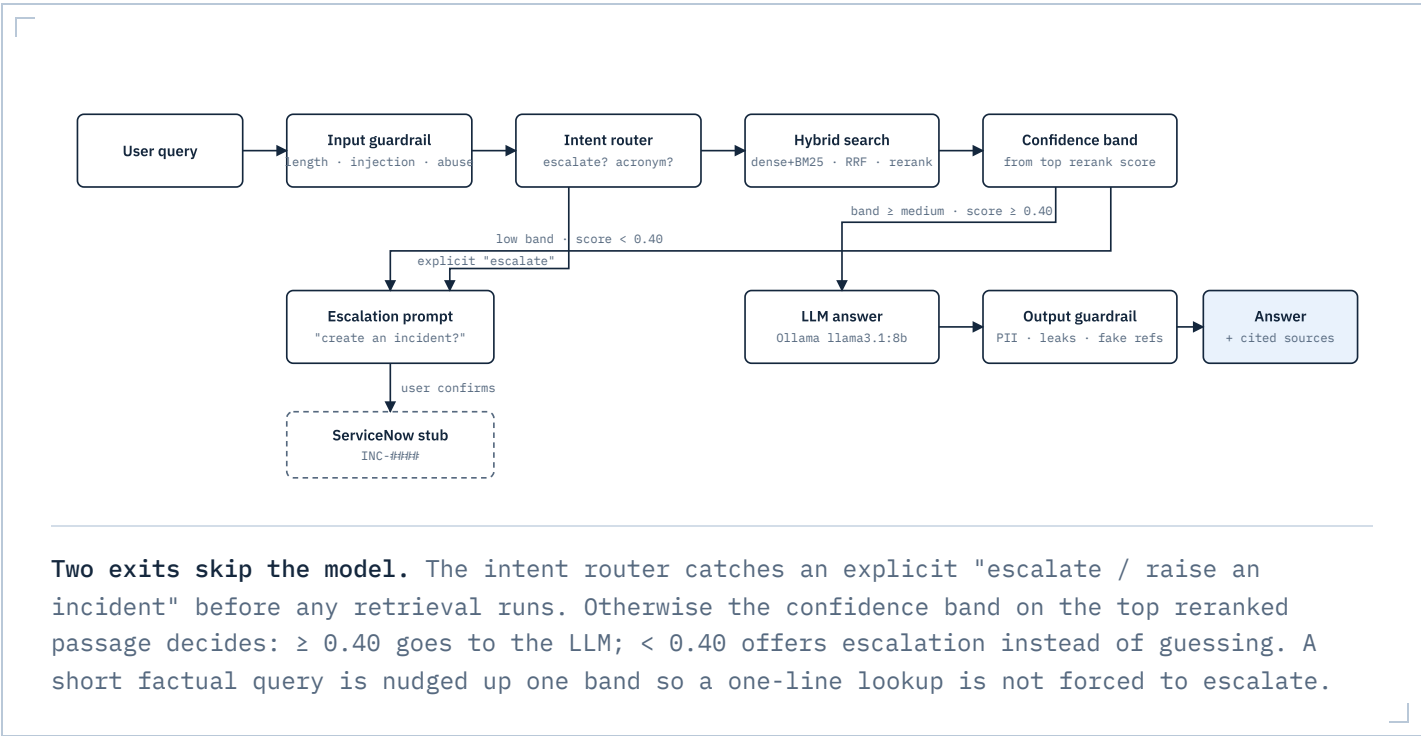
§2 System architecture



COMPONENT	MODULE	RESPONSIBILITY
Chat UI	<code>streamlit_app.py</code>	ChatGPT-style front end: conversation sidebar, message history, rendered source citations and figure images. Talks to the API over HTTP only.
API & lifespan	<code>api/main.py</code>	<code>/chat</code> , <code>/upload</code> , <code>/escalate</code> , <code>/conversations*</code> , <code>/documents</code> , <code>/stats</code> , <code>/health</code> . Starts the file watcher and builds BM25 indexes on boot.
Orchestrator	<code>chatbot/chain.py</code>	The RAG chain: guardrail checks, intent detection, follow-up query rewriting, hybrid search, confidence scoring, prompt assembly, LLM call, output validation.
Retrieval	<code>retrieval/hybrid_search.py</code> <code>retrieval/bm25_index.py</code> <code>retrieval/confidence_scorer.py</code>	Per-collection dense + keyword search, reciprocal rank fusion, cross-encoder rerank, exact record-id pinning, intent-driven promotion of figures / summaries / transcripts, confidence banding.
Vector store	<code>vectorstore/chroma_store.py</code>	ChromaDB <code>PersistentClient</code> , cosine HNSW, four collections. Upsert, filtered query, delete-by-source helpers.
Ingestion	<code>ingestion/pdf_ingester.py</code> <code>ingestion/excel_ingester.py</code> <code>ingestion/video_ingester.py</code> <code>ingestion/file_watcher.py</code>	Type-specific extraction and chunking, shared PII scrub + sensitivity filter, batched embedding, upsert, BM25 rebuild, registry updates. Auto-triggered by the watcher.
Security	<code>security/pii_scrubber.py</code> <code>security/sensitivity_tagger.py</code> <code>guardrails/*.py</code>	Presidio + spaCy PII redaction, keyword sensitivity tagging, and the input / output guardrails.
Registry	<code>registry/document_registry.py</code> <code>chatbot/conversation_store.py</code>	SQLite: document lifecycle (<code>doc_id</code> , filename, type, version, status, chunk count) and durable chat history that survives restarts.
Escalation	<code>chatbot/servicenow_handler.py</code>	Builds an incident payload from the conversation and returns a dummy <code>INC<timestamp></code> . No integration — a placeholder for a real connector.
Desktop shell	<code>desktop_app.py</code>	Optional pywebview wrapper that packages the Streamlit UI as a native window.

§3 Request lifecycle

A question passes through a fixed sequence of gates. Two of them — an explicit escalation request and a low-confidence retrieval — can divert the request away from the language model entirely.

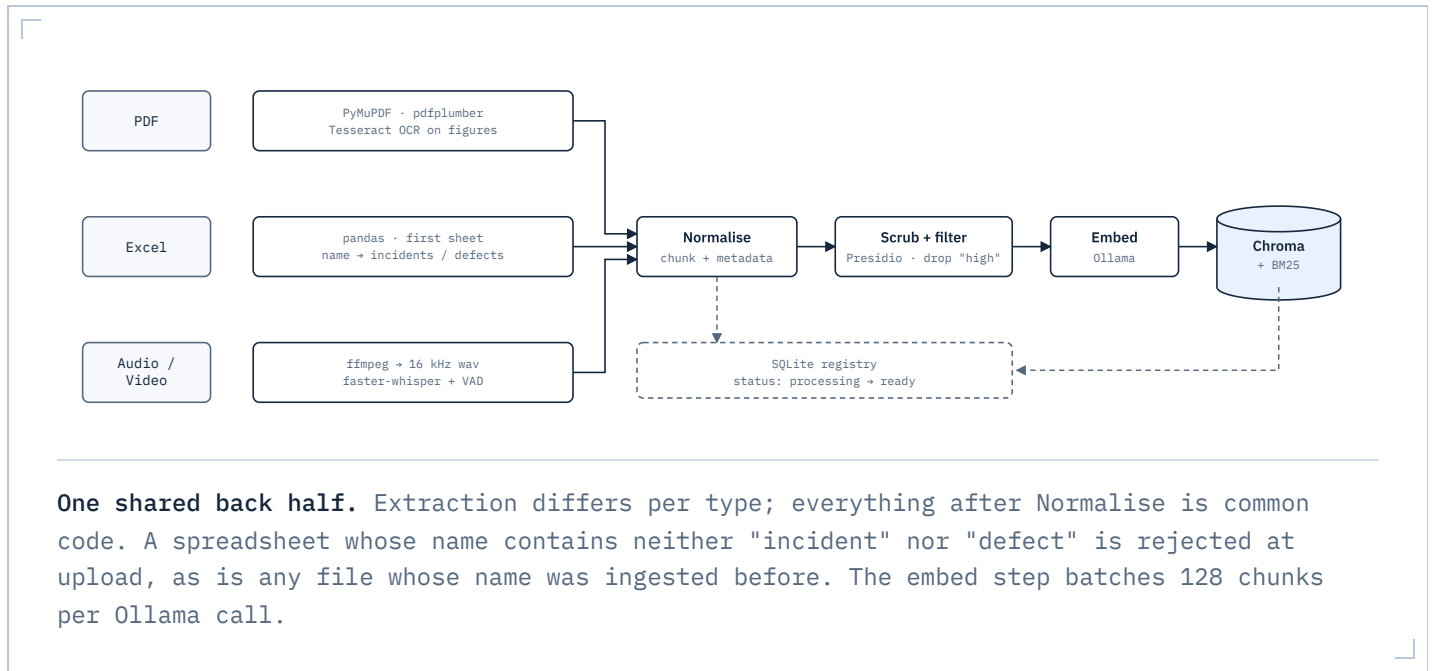


0 · pending check	If the previous turn offered escalation, a yes / no here is handled before anything else.
1 · input guardrail	Empty, over-length (> 500 chars), prompt-injection patterns, or abuse → rejected with a plain message.
2 · intent router	Escalation phrasing → confirmation prompt. Acronym / "what does X stand for" → BM25 definition lookup, no LLM.
3 · query rewrite	Short follow-ups are enriched with the entity or topic from the previous turn (heuristic, no model call).
4 · hybrid search	Dense + BM25 per collection → RRF → cross-encoder rerank → top 5. Confidence = normalized score of rank 1.
5 · escalation gate	Low band → offer to raise an incident. The original query is held for the payload.

6 · generation	Context block + conversation history + banded instructions → llama3.1:8b at temperature 0.1.
7 · output guardrail	Redact PII, strip internal identifiers, flag error numbers absent from the context, truncate to the token cap.

§4 Ingestion pipelines

Three source types share one back half. Each has its own extractor; from the chunking step onward the path is identical — scrub, filter, embed, index. The target collection is chosen by file type, and for spreadsheets by file name.



PDF

Text blocks, tables (pdfplumber), and embedded raster figures. Figures are detected by image size and page position — not by caption — then run through Tesseract so diagram labels become searchable text. Figure images are saved and served back with the answer.

Excel

Generic single-sheet reader: first row is the header, columns are arbitrary. One chunk per row plus a sheet-overview chunk for "how many" questions. Every column is kept as `col_*` metadata. Routed to `incident_tickets` or `defect_records` by file name.

Audio / video

ffmpeg extracts a 16 kHz mono track; faster-whisper ("medium" + voice activity detection) transcribes it. Segments are merged into ~60-second / 700-character windows, each carrying its timestamp, plus one full-transcript overview chunk. Sidecar `.srt` / `.json` are written too.

§5 Retrieval & confidence

Retrieval runs every collection in parallel as independent ranked lists, then fuses them so a large collection cannot outweigh a small one by sheer volume. Fusion feeds a cross-encoder, which is the score that actually decides the answer.

dense	Top 20 per collection from ChromaDB, cosine similarity on nomic-embed-text vectors.
sparse	Top 20 per collection from a per-collection BM25 Okapi index, rebuilt after every ingest.
fusion	Reciprocal rank fusion, $k = 60$, ranked <i>within</i> each list — every list's rank 1 contributes $1/(k+1)$. Fused to 10, plus each collection's own best candidate.
rerank	cross-encoder/ms-marco-MiniLM-L-6-v2 scores every query-passage pair; top 5 survive.
exact id	A bare record token in the query (e.g. SNOW8729267) is looked up by row_key and pinned to rank 0.
promotions	Figure-intent, aggregate-intent ("how many"), and transcript-intent queries each force their matching chunk type into the result set if absent.

BAND	SCORE	BEHAVIOUR
high	≥ 0.75	Answer directly and completely from context.
medium	$0.40 - 0.75$	Answer, but instructed to extract inline detail and note gaps.
low	< 0.40	No LLM call — offer to raise an incident instead.

§6 Security & guardrails

Input guardrail

Rejects empty input, queries over 500 characters, prompt-injection patterns ("ignore previous instructions", "you are now...", [system]), and abusive language — each with a specific message.

PII scrub (ingest)

Presidio + spaCy (en_core_web_sm, trimmed pipeline) redact PERSON, EMAIL_ADDRESS, PHONE_NUMBER, IP_ADDRESS, US_SSN, CREDIT_CARD, IBAN_CODE before anything is embedded. Runs as one batched NLP pass over all chunks.

Output guardrail

Regex-redacts email / phone / IP / SSN from the model's reply, replaces leaked internal terms (chromadb, doc_id, file paths) with [internal], flags error numbers not present in the retrieved context, and truncates to the response token cap.

Sensitivity filter

Keyword tagging (salary, payroll, confidential, password, credential...). A chunk tagged "high" is dropped — never embedded, never retrievable. Figure chunks are flagged as possibly containing PII in the image itself.

URL and LOCATION are deliberately *not* redacted: URL redaction removes document links and hurts retrieval; LOCATION rarely carries real PII in ERP material and is costly to detect.

§7 Data stores

COLLECTION	FED BY	CHUNK TYPES	KEY METADATA
pdf_chunks	PDF ingester	text, table, figure	page, section, chunk_type, image_path
incident_tickets	Excel, name contains "incident"	excel_row, sheet_summary	row_number, row_key, col_*
defect_records	Excel, name contains "defect" / "bug"	excel_row, sheet_summary	row_number, row_key, col_*
video_transcripts	Audio / video ingester	transcript_segment, transcript_summary	start_sec, end_sec, start_time, segment_index

Vector index	ChromaDB PersistentClient at vectorstore/chroma_db/, cosine HNSW. Every collection also has a pickled BM25 index under vectorstore/bm25_indexes/.
document_registry.db	One row per ingested file: doc_id, filename, source type, version, status, chunk count, timestamp. Drives duplicate-name rejection.
conversations.db	Conversations and messages with confidence, sources and images. The chat window is rehydrated from here after a restart.
data/ layout	uploads/ (watched), figures/ (served), transcripts/ (sidecars). Nothing here is authoritative — the vector store and registry are.

§8 Technology stack

Models (local)

Ollama — llama3.1:8b, nomic-embed-text (768-dim). sentence-transformers 4.1 cross-encoder. spaCy en_core_web_sm 3.8. faster-whisper 1.1.

Ingestion

pymupdf 1.25.5, pdfplumber 0.11.6, pytesseract 0.3.13 + Tesseract binary, pandas 2.3.3 / openpyxl 3.1.5, ffmpeg binary, watchdog 6.0.

Retrieval & store

chromadb, rank-bm25 0.2.2. Orchestration glue from langchain 0.3.25 / langchain-community / langchain-ollama.

Service & UI

fastapi 0.115 / uvicorn 0.34, streamlit 1.45, pywebview 6.2, SQLAlchemy 2.0, presidio-analyzer / anonymizer 2.2.356.

External binaries required on the host: **Ollama**, **Tesseract OCR**, **ffmpeg**. Everything else installs from requirements.txt.

§9 Known limits & next steps

LIMIT	DETAIL	DIRECTION
CPU-only inference	Embedding is ~83% of ingestion wall-clock; generation is the slowest step at query time. Concurrency gave no gain on shared cores.	A GPU for Ollama is the single biggest speedup. Batch size and concurrency are already env-configurable for that day.
Vector-drawn diagrams	Only embedded raster images are extracted from PDFs. Diagrams drawn as vector paths are not captured as figures.	Cluster drawing-operation bounding boxes and render those regions to raster before OCR.
Table-of-contents noise	Dotted TOC lines can rank as high-confidence sources.	Filter leader-dot / page-number-only chunks at clean time.
Escalation is a stub	No real ticketing integration; the incident id is synthetic.	Replace <code>servicenow_handler</code> with a real connector behind the same interface.
Single tenant	No authentication on the API; one shared conversation store; assumes a trusted local network.	Add auth and per-user scoping before any multi-user deployment.
Transcription quality	Accuracy tracks source audio; quiet or noisy recordings degrade badly even at the "medium" model.	Surface a confidence signal per transcript; allow a larger model where the host can afford it.